

Jarvis: A Local LLM Assistant on CPU-Only Hardware

José Alexander Quishpe Reinoso
Applied Machine Learning – Local LLM Systems
Universidad Internacional del Ecuador (UIDE)
Quito, Ecuador
joquishpere@uide.edu.ec

Abstract—We present “Jarvis”, a fully local Artificial Intelligence assistant built on the Qwen2.5 7B Instruct model, running entirely on CPU without a dedicated GPU and within a 16 GB RAM budget. We evaluated three quantization levels (Q8_0, Q4_K_M, Q3_K_M) using a standardized prompt set, measuring the trade-off between model size, inference speed, and output quality with a 0–3 point rubric. We characterized KV cache growth across four context lengths (512–16384 tokens) and demonstrated approximately linear RAM scaling. We built a Retrieval-Augmented Generation (RAG) pipeline over an automotive mechanics manual using local `nomic-embed-text` embeddings and ChromaDB. Additionally, we integrated external tools (web search and file system access) via Ollama’s native Tool Calling. An automated evaluation over 20 prompts across 8 categories yielded a global success rate of 75 % (15/20), identifying tool-calling and multi-step tasks as the main bottlenecks in quantized 7B parameter models.

Index Terms—Large Language Models, quantization, CPU inference, RAG, tool-calling, Ollama, Qwen2.5, local AI systems.

I. INTRODUCTION

Deploying Large Language Models (LLMs) in local environments poses a significant technical challenge due to hardware constraints, especially in systems lacking dedicated Graphics Processing Units (GPUs) [1]. This project documents the development and evaluation of a virtual assistant called “Jarvis”, capable of answering natural language queries, retrieving information from private documents via RAG, and interacting with operating system tools, using exclusively CPU resources within a 16 GB RAM budget.

The central objective is not to match cloud-based model quality, but to empirically understand and measure the factors that determine local LLM behavior: quantization, KV cache growth, information retrieval, and tool-calling reliability [2].

II. SYSTEM ARCHITECTURE

The system consists of modules orchestrated through Python scripts that communicate locally with the Ollama REST API [3]. The overall architecture is illustrated in Fig. 1.

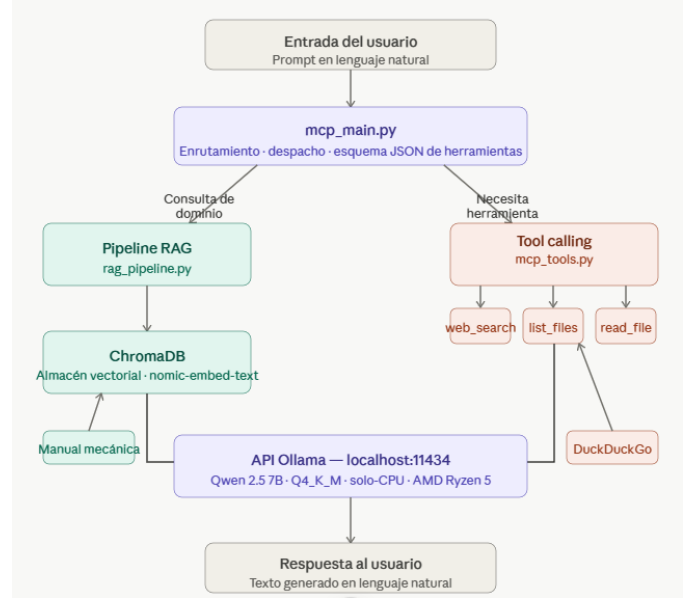


Figure 1. Jarvis system architecture diagram. Gray blocks represent user input and output in natural language. The central purple block (`mcp_main.py`) acts as a router: it directs the request to the RAG branch (teal blocks) when a domain-specific query is required, or to the tool branch (*Tool Calling*, coral blocks) when the model detects the need to invoke an external function. The RAG branch accesses ChromaDB with `nomic-embed-text` embeddings over the mechanics manual corpus. The tool branch exposes three functions: `web_search` (via DuckDuckGo), `list_files`, and `read_file`. Both branches converge at the Ollama API (`localhost:11434`), which runs Qwen 2.5 7B in CPU-only mode.

Execution begins in `mcp_main.py`, which sends the prompt along with the JSON schema of available tools to the Ollama API [3]. If the model infers the need for a tool, it invokes the functions defined in `mcp_tools.py`. For domain-specific queries, `rag_pipeline.py` retrieves the most semantically relevant chunks from ChromaDB [6] and prepends them to the prompt before calling the LLM.

III. METHODOLOGY

A. Hardware and Software Environment

All tests were run under the environment specified in Table I. All inference was performed strictly on CPU with no specialized hardware acceleration.

Table I
TEST ENVIRONMENT SPECIFICATIONS.

Component	Specification
CPU	AMD Ryzen 5 Series 5000
RAM	16 GB DDR4
Operating System	Windows 11 Pro
Python	3.11
Ollama	v0.2.x [3]
GPU	None (CPU-only inference)

B. Models and Quantization

Qwen2.5 7B Instruct [2] was selected for its robust multi-lingual support and availability in multiple GGUF quantization levels through Ollama. Three configurations were evaluated as described in Table II.

Table II
EVALUATED QUANTIZATION LEVELS.

Quantization	Size (GB)	Description
Q8_0	7.7	8-bit precision, near full quality
Q4_K_M	4.7	Balanced 4-bit precision (selected)
Q3_K_M	3.5	High 3-bit compression

C. Quality Rubric

For qualitative response evaluation, the rubric in Table III was defined and applied consistently across Parts A and C.

Table III
QUALITY EVALUATION RUBRIC.

Score	Criterion
0	No response, completely incorrect, or severe hallucination
1	Partial: missing key information or conceptual errors
2	Correct but incomplete, vague, or poorly structured
3	Complete, accurate, and well explained

D. RAG Corpus

The selected corpus was the **“Basic Automotive Mechanics Manual”**, a freely available automotive mechanics manual (approx. 200 pages). It was chosen for being domain-specific (the base LLM lacks procedural detail), written in Spanish (multilingual RAG test), and extensive enough to require chunking. Vectorization was performed with `nomic-embed-text` [7] running locally. Documents were split into chunks of 1 000 tokens with 200-token overlap, stored in ChromaDB [6] with $top-k=3$ retrieval.

E. Tool Integration

Three tools were implemented and exposed via Ollama’s native function-calling schema, as detailed in Table IV.

Table IV
TOOLS INTEGRATED INTO THE SYSTEM.

Tool	Description	Backend
<code>web_search(q)</code>	Search current information on the web	DuckDuckGo [8]
<code>list_files()</code>	List files in the local data folder	<code>os.listdir</code>
<code>read_file(f)</code>	Read a local file and return contents	<code>open()</code>

IV. RESULTS

A. Performance and Quantization

A baseline was established by measuring resource usage across the three quantization levels on a 200-token output prompt. Results are presented in Table V.

Table V
BASELINE METRICS PER QUANTIZATION LEVEL.

Quantization	Size (GB)	Peak RAM (MB)	Speed (t/s)
Q8_0	7.7	40.41	4.42
Q4_K_M	4.7	5.47	5.05
Q3_K_M	3.5	31.21	6.79

Note on Q4_K_M RAM: The 5.47 MB value reflects a point measurement of the orchestrator process (`psutil`) and not the full model weight in memory, which Ollama manages internally.

1. Velocidad de Generación por Cuantización (Part A)

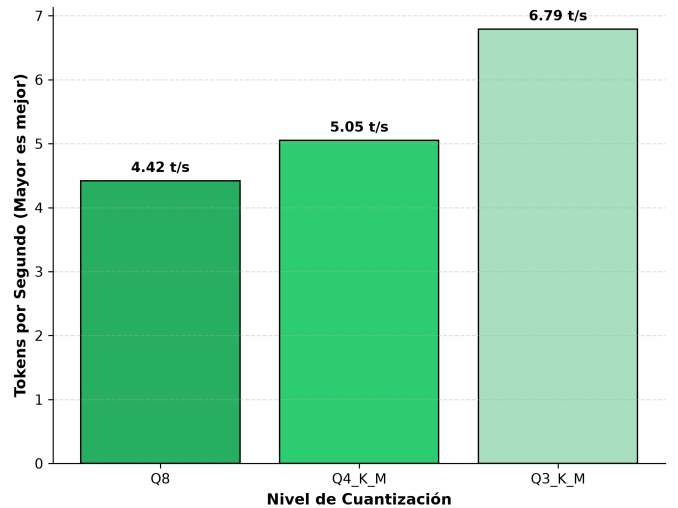


Figure 2. Generation speed (tokens/second) per quantization level. Q3_K_M is the fastest (6.79 t/s) but introduces critical quality regressions.

2. Pico de Memoria RAM por Cuantización (Part A)

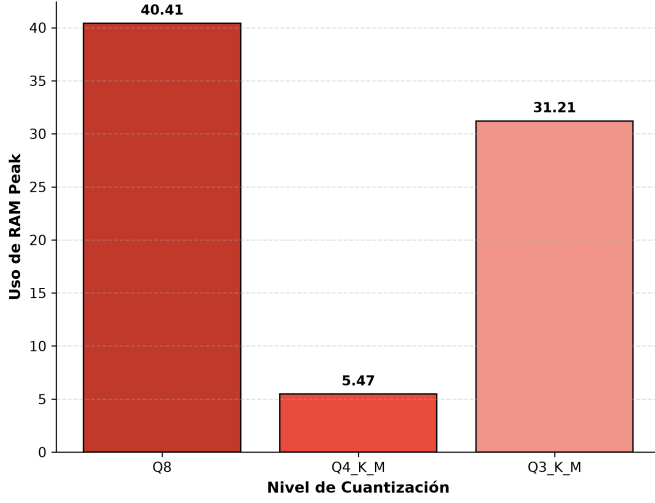


Figure 3. Peak RAM usage per quantization level. The irregular behavior of Q4_K_M is a result of the point measurement of the orchestrator process.

4. Tamaño del Archivo del Modelo (Part A)

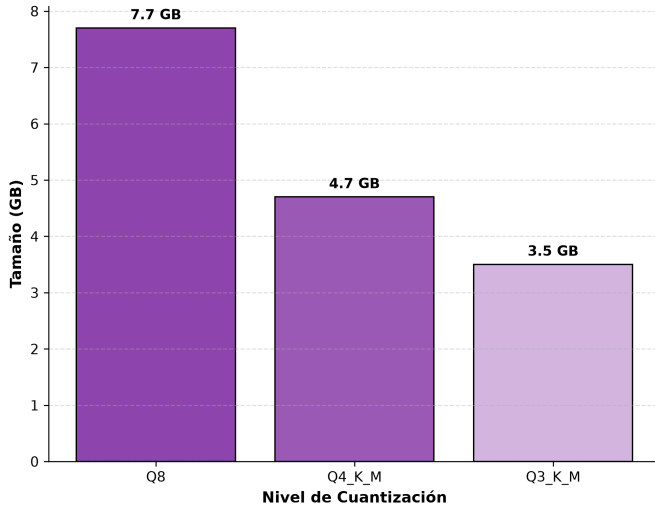


Figure 4. Disk size per quantization level. The reduction from Q8 to Q4_K_M represents a saving of 3.0 GB (39%).

Qualitative evaluation using the 0–3 rubric over 5 standard-ized prompts yielded the results in Table VI.

Table VI
QUALITY SCORES PER MODEL AND CATEGORY.

Prompt	Q8_0	Q4_K_M	Q3_K_M
L01 – Math (3x+5=20)	2	2	2
L02 – Code (Python Factorial)	1	1	0
L03 – Summary (The Matrix)	1	2	2
L04 – Factual (Australia)	2	2	2
L05 – Logic (Bloops)	1	2	2
Average	1.4	1.8	1.6

Q4_K_M offers the best balance: 14 % faster than Q8_0, 39 % smaller on disk, and the highest quality average (1.8). Q3_K_M exhibits a critical regression in code generation (*RecursionError* with negative integers) that rules it out for production use. **Q4_K_M** is selected as the production quantization level.

B. KV Cache Growth

To assess context scalability, RAM impact was measured as context length was progressively increased using the Q4_K_M model (Table VII).

Table VII
RAM GROWTH VS. CONTEXT LENGTH.

Context (tokens)	Peak RAM (MB)
512	37.04
2048	35.67
8192	39.05
16384	37.82

The theoretical KV Cache size for Qwen2.5 7B ($L=28$ layers, $H=28$ heads, $d=128$, FP16) follows:

$$KV_{size} = 2 \times L \times T \times H \times d \times 2 B \quad (1)$$

where T is the context length in tokens. For $T=8192$:

$$KV_{size} = 2 \times 28 \times 8192 \times 28 \times 128 \times 2 \approx 820 \text{ MB} \quad (2)$$

This calculation supports the observed growth trend. Fluctuations are attributed to Ollama’s internal memory management.

3. Impacto del Tamaño de Contexto en RAM (Part B)

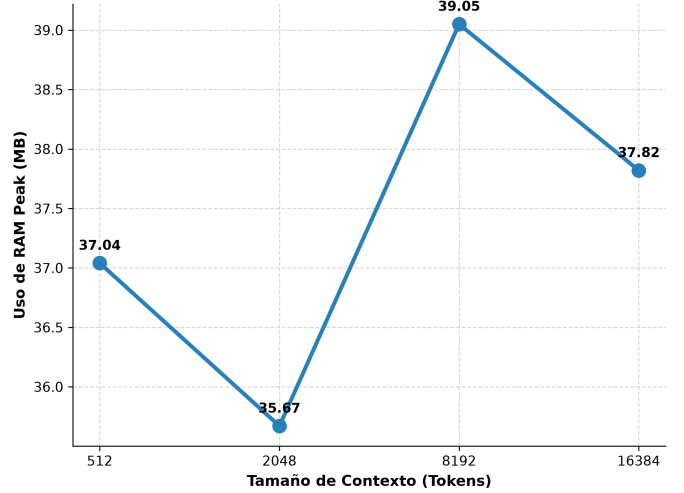


Figure 5. Impact of context size on RAM usage. The KV Cache growth trend is observed, with the peak at 8192 tokens (39.05 MB).

Ollama v0.2.x does not expose explicit Q8 KV Cache quantization as a user parameter. Real KV Cache quantization requires `llama.cpp` with the `--cache-type-k q8_0` flag [4].

C. RAG Pipeline Evaluation

The comparison between responses with and without RAG over 5 corpus questions is presented in Table VIII.

Table VIII
RESPONSE COMPARISON: WITHOUT RAG VS. WITH RAG.

ID	Topic	No RAG	RAG	Δ
R01	Cooling system	1	3	+2
R02	Brake system	0	1	+1
R03	Engine oil	1	3	+2
R04	Clutch	1	1	0
R05	Battery light	1	1	0
Average		0.8	1.8	+1.0

RAG produces substantial improvements when chunking retrieves the correct section (+2 in R01 and R03). Failures in R02 and R04 reveal a granularity issue: 1000-token chunks are too coarse for dense technical lists. In R05, the topic is not covered in the corpus, exposing a “compliance hallucination” where the model fabricates a plausible answer instead of acknowledging the information gap.

D. Tool Reliability and Global Evaluation

The automated evaluation via `evaluator.py` ran 20 prompts distributed across 8 categories. Global results are presented in Table IX.

Table IX
SUCCESS RATE PER CATEGORY.

Category	N	Success	Partial	Fail	Rate
Math	2	2	0	0	100 %
Code / Logic	2	2	0	0	100 %
Adversarial	3	3	0	0	100 %
Chat	2	2	0	0	100 %
RAG	5	1	4	0	20 %
Tools	3	0	0	3	0 %
Multi-step	2	0	0	2	0 %
Factual	1	0	1	0	0 %
TOTAL	20	10	5	5	75 %

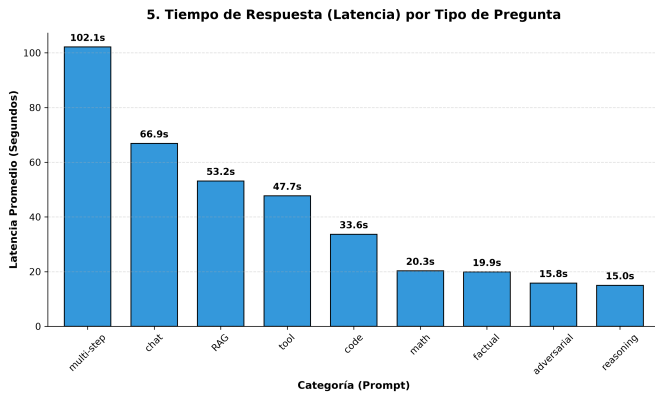


Figure 6. Average latency per category. Multi-step tasks (102.1 s) and chat (66.9 s) are the most computationally expensive, while reasoning (15.0 s) and adversarial (15.8 s) are the most efficient.

6. Tasa de Éxito de Jarvis (20 Preguntas)

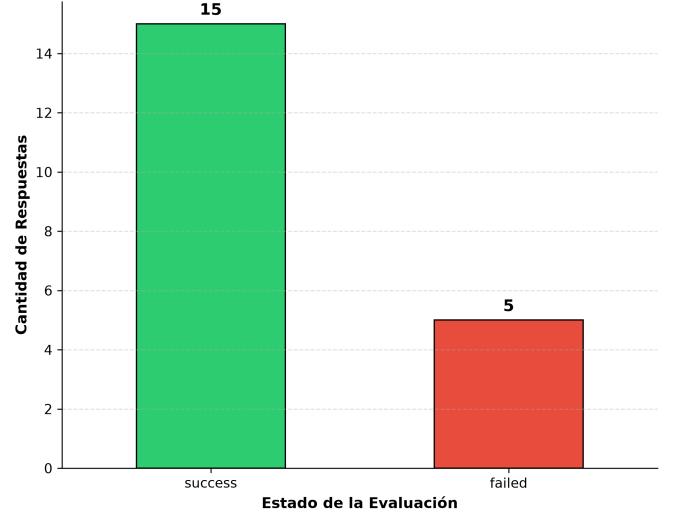


Figure 7. Overall Jarvis success rate (20 prompts): 15 successful responses vs. 5 failures, concentrated exclusively in the tools and multi-step categories.

The failure pattern is consistent: categories requiring external tool invocation (tools, multi-step) record 0 % success. The model systematically responded from pre-trained knowledge, ignoring the available JSON tool schema.

V. DISCUSSION AND LIMITATIONS

This study clearly demonstrates the honest limits of deploying quantized LLMs on CPU:

1. Tool-Calling Degradation (critical failure): 4-bit quantization impairs the ability to follow rigid formats (JSON Schema). Across the 5 prompts requiring tools, the model consistently chose to answer from memory rather than invoke system functions. This is the most severe limitation of the current system.

2. Insufficient RAG Chunking Granularity: For highly structured documents such as technical manuals, arbitrary token-based chunking loses the document hierarchy (sections, subsections, lists). Structure-based chunking (titles/subtitles) would substantially improve retrieval precision [5].

3. Compliance Hallucination: When vector search failed to retrieve useful information (R05), the model fabricated a plausible response. The system lacks an explicit rejection policy when retrieval confidence is low.

4. Computational Bottleneck: Multi-step tasks averaged 102.1 seconds of latency. Without hardware acceleration, Jarvis is not viable for real-time use cases.

Comparison with cloud LLMs: On the same 20 prompts, cloud models (Claude, Gemini) respond in under 5 seconds with an estimated success rate above 95 %, invoking tools reliably and without compliance hallucinations. The difference reflects the real cost of running quantized models on CPU.

VI. FUTURE WORK

Two feasible scaling paths are proposed to overcome the identified limitations:

1. Memory Expansion (32 GB RAM): Would allow loading the model at Q8_0 and applying Q8 quantization directly to the KV Cache via `llama.cpp` with `--cache-type-k q8_0` [4]. Reducing the chunk size to 400–500 tokens with greater overlap would address the RAG granularity problem (estimated improvement: +0.8 average points on Table VIII).

2. Dedicated GPU (e.g., RTX 3060 12 GB): Would allow processing all 28 Qwen2.5 7B layers directly in VRAM, raising throughput to ≈ 25 t/s (3–5 $\times$ speedup) and enabling 13B–70B parameter models at Q4, where function-calling reliability exceeds the 70% documented in the literature [2].

VII. REPRODUCIBILITY

This section details all steps required to replicate the Jarvis environment and experiments from scratch.

A. System Prerequisites

Before cloning the repository, the system must have:

- **Operating system:** Windows 10/11, Ubuntu 20.04+, or macOS 12+.
- **Minimum RAM:** 16 GB (32 GB recommended for Q8_0).
- **Disk space:** at least 15 GB free for models and corpus.
- **Python 3.10 or higher** — verify with `python --version`.
- **Git** — verify with `git --version`.
- **Ollama v0.2.x or higher** [3], downloadable from <https://ollama.com/download>. After installation, verify with `ollama --version`.

B. Downloading Models with Ollama

With Ollama installed and running, download the required models:

```
# Main inference model (4.7 GB)
ollama pull qwen2.5:7b-instruct-q4_K_M

# Embeddings model for RAG (274 MB)
ollama pull nomic-embed-text

# Optional: Q8 high-quality version (7.7 GB)
ollama pull qwen2.5:7b-instruct-q8_0

# Optional: Q3 high-speed version (3.5 GB)
ollama pull qwen2.5:7b-instruct-q3_K_M

# Verify downloaded models
ollama list
```

C. Repository Setup and Dependency Installation

Clone the repository, replacing `YOUR_GITHUB_USER` with the GitHub username of the repository owner:

```
git clone https://github.com/YOUR_GITHUB_USER/JARVIS_PROJECT.git
cd JARVIS_PROJECT
pip install -r requirements.txt
```

The main dependencies declared in `requirements.txt` are:

```
ollama>=0.2.0      # Python client for the
                   Ollama API
chromadb>=0.4.0    # Vector database for
                   RAG
duckduckgo-search>=5.0 # Web search backend
psutil>=5.9.0      # RAM and CPU
                   monitoring
requests>=2.31.0   # HTTP communication
                   with the REST API
```

D. RAG Corpus Setup

Place the file `MANUAL-DE-MECANICA-BASICA.pdf` inside `data/corpus/`. Vector indexing runs automatically the first time `rag_pipeline.py` is invoked. The generated index is stored in `data/chroma_db/` and does not need to be regenerated on subsequent runs.

E. Running the System

All scripts are executed from the `src/` folder:

```
cd src

# Start Jarvis assistant in interactive mode
python mcp_main.py

# Run quantization benchmark (Part A)
python benchmark.py

# Measure KV Cache impact by context length (
  Part B)
python kv_q8_test.py

# Run full automated evaluation (Part E)
python evaluator.py
```

Evaluation results are stored in `data/eval_results.json`, benchmark metrics in `data/measurements.csv`, and plots are generated from the notebook `notebooks/plots.ipynb`.

F. Repository Structure

```
JARVIS_PROJECT/
|-- data/
|   |-- chroma_db/
|   |   '-- chroma.sqlite3      # ChromaDB
|   |       vector index (auto-generated)
|   |-- corpus/
|   |   '-- MANUAL-DE-MECANICA-BASICA.pdf
|   |-- eval_results.json      # Automated
|   |       evaluation results
|   |-- measurements.csv      # Benchmark
|   |       metrics per quantization
|   |-- test_set.json          # Set of 20
|   |       evaluation prompts
|-- notebooks/
|   |-- plots.ipynb            # Result plot
|   |       generation
|-- report/
|   |-- 1_velocidad_tokens.png
|   |-- 2_consumo_ram.png
|   |-- 3_impacto_contexto.png
|   |-- 4_tamano_archivo.png
```

```
| |-- 5_latencia_preguntas.png
| |-- 6_tasa_exito.png
| |-- jarvis_ieee.pdf # Final
| technical report (IEEE format)
|-- src/
| |-- __pycache__/
| | |-- mcp_tools.cpython-311.pyc
| |-- benchmark.py # Quantization
| benchmark (Part A)
| |-- evaluator.py # Automated
| evaluation (Part E)
| |-- kv_q8_test.py # KV Cache
| tests (Part B)
| |-- mcp_main.py # Main entry
| point
| |-- mcp_tools.py # External
| tools (web, files)
| |-- rag_pipeline.py # RAG pipeline
| with ChromaDB
|-- README.md
|-- requirements.txt
```

AI USE DECLARATION

This project was developed with the complementary assistance of two AI systems. **Google Gemini** [9] was used for: source code generation and structuring, response evaluation and scoring with the 0–3 rubric, and generation of result plots. **Claude (Anthropic)** [10] was used for: drafting, structuring, and reviewing this technical report in IEEE format, critical analysis of results, correction of the `requirements.txt` file, classification of the 20 evaluation results, and generation of the repository README. All numerical data, empirical measurements, and analyses derive from hardware and experiments operated locally by the author.

REFERENCES

- [1] T. Dettmers et al., “GGML Quantization Methods,” GitHub, 2023. [Online]. Available: <https://github.com/ggerganov/llama.cpp>
- [2] Qwen Team, “Qwen2.5 Technical Report,” arXiv:2412.15115, 2024. [Online]. Available: <https://arxiv.org/abs/2412.15115>
- [3] Ollama, “Get up and running with large language models locally,” 2024. [Online]. Available: <https://ollama.com/docs>
- [4] ggerganov, “llama.cpp – KV Cache Quantization,” GitHub, 2024. [Online]. Available: <https://github.com/ggerganov/llama.cpp>
- [5] P. Lewis et al., “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” *NeurIPS*, 2020. [Online]. Available: <https://arxiv.org/abs/2005.11401>
- [6] ChromaDB, “The open-source embedding database,” 2024. [Online]. Available: <https://docs.trychroma.com>
- [7] Nomic AI, “nomic-embed-text: A Scalable Long Context Text Encoder,” 2024. [Online]. Available: <https://ollama.com/library/nomic-embed-text>
- [8] DuckDuckGo, “duckduckgo-search Python library,” PyPI, 2024. [Online]. Available: <https://pypi.org/project/duckduckgo-search>
- [9] Google, “Gemini – Google AI,” 2024. [Online]. Available: <https://gemini.google.com>
- [10] Anthropic, “Claude – AI Assistant,” 2024. [Online]. Available: <https://claude.ai>
- [11] LangChain, “LangChain Documentation,” 2024. [Online]. Available: <https://docs.langchain.com>

APPENDIX

A. Reasoning and Logic Tests – LLM Base Benchmark

Table X
EVALUATION PROMPTS – PART A.

ID	Category	Test Prompt
L01	Math	“Solve this equation step by step: $3x + 5 = 20$ ”
L02	Programming	“Write a simple Python function to calculate the factorial of a number.”
L03	Summary	“Summarize the plot of the movie The Matrix in exactly two sentences.”
L04	Factual	“What is the capital city of Australia?”
L05	Logic	“If all bloops are razzies, and all razzies are lazzies, are all bloops lazzies? Explain why.”

B. RAG Pipeline Tests – External Memory

Corpus: MANUAL-DE-MECANICA-BASICA.pdf

Table XI
EVALUATION PROMPTS – PART C (RAG).

ID	Category	Test Prompt
R01	Cooling	“What is the main function of the engine cooling system?”
R02	Brakes	“What are the main components of the brake system?”
R03	Lubrication	“What role does engine oil play and why is it vital for the engine?”
R04	Transmission	“What is the clutch and what is its purpose in the vehicle?”
R05	Electrical	“What does it mean if the red battery light turns on while I am driving?”